Introduktion till C++

Uppgift 3.2 - Kasino med klasser

Uppgiftens målsättning

I uppgiften introduceras objektorientering där funktionalitet och data ramas in i objekt. Under tidigare uppgifter har den studerande byggt upp en struktur av olika spel, menyer och funktionalitet. Nu skall programmet refaktoreras för att ge övning på de objektorienterade koncepten och hur det kan vara hjälpsamt vid behov av återanvändbarhet.

Uppgift

Uppgiften består av tre delar:

- Refaktorering
    - Implementationen skall refaktoreras för att använda klasser. Detta för att öva på att rama in data och funktionalitet
- Variation på spel
    - Den andra delen belyser styrkan i klasser för att skapa två (eller flera) varianter av logiskt grupperade data. För att se exempel och ge övning på detta används ett existerande spel som uppdateras med funktionalitet som gör att varianter av spelet enkelt kan konstrueras endast genom att initialisera spelet med olika data.
- Spelarens namn
    - Vi skall utöka kasinot så att spelaren anger sitt namn vid start. För att öva på förståelsen för programmets flöde skall namnet användas vid flera separata situationer.

Refaktorering

Skriv om kasino-spelet så att det använder sig av klasser och objekt. Det ska finnas en klass för varje spel uppdelad i en header-fil och en källkodsfil med samma namn (förutom filtillägget, såklart). Varje klass har en medlemsvariabel som håller reda på hur mycket man har vunnit eller förlorat på just det spelet.

Gör även en klass för hela kasinot som i sin tur innehåller de olika spelen. Här finns en medlemsvariabel för varje spel.

Medlemsvariabler ska vara privata och de som behöver läsas utifrån ska ha get-funktioner. Om variabeln inte ska ändras kan den vara const. Om alla instanser av klassen delar den kan den vara static. Alla medlemsfunktioner (inklusive alla get-funktioner) som inte ändrar på några medlemsvariabler ska deklareras som const.

Ta för vana att tänka på vad som skickas in till en konstruktor och om en explicit definierad konstruktor faktiskt behövs. Om en konstruktor tar parametrar, fundera då på om parametern behövs eller om den kan räknas ut, till exempel från övriga parametrar. I C++ genereras en konstruktor så om vi inte explicit behöver initialisera data i klassen, så behöver vi sannolikt inte definiera en konstruktor.

Variation på spel

Modifiera det första spelet vi gjorde, gissa numret, så att vi kan lägga till ett extra spel med andra satsningsgränser genom att ändra i dess klass. Konstruktorn behöver alltså ta in min- och max-värden för hur mycket man får satsa vid bordet. På så vis kan ni skapa två bord med olika gränser för hur mycket pengar man får/måste **satsa**, ex *Low Stakes* och *High Stakes*. De olika gränserna för vad man får tjäna på bordet ska ni räkna ut från min- och/eller max-gränsen. Här fyller konstruktorn en funktion, den tar emot inställningar som sparas internt i instansen så att två instanser av samma klass kan ha olika regler.

Eftersom de har olika min- och max-gränser måste dessa presenteras så att spelaren vet hur mycket pengar de minst måste satsa respektive högst får satsa. Dessa två olika bord är då bara två olika variabler där varje instans har initialiserats med olika värden skickade till konstruktorn. Detta innebär att de hanteras precis likadant utanför klassen och håller internt reda på sina olikheter.

Om man inte längre kan spela på bordet för att man inte har nog med pengar för att satsa minimumbeloppet så ska spelaren komma tillbaka till huvudmenyn.

Spelarens namn

Spelaren skall i början av spelet få ange sitt namn. Namnet ska valideras så att det bara **innehåller bokstäver** samt är **mellan två och 16 tecken långt**.

Detta namn ska sedan **användas genom spelet** i konversation med spelaren. Det innebär att det inte räcker att presentera namnet i lobbyn, utan det bör användas på olika passande delar av spelet och olika spelbord.

Namnet ska läsas in till en char array. För att kunna spara 16 tecken långa namn måste arrayen vara 17 tecken eftersom strängar alltid slutar med 0. Lättast läses strängen in med cin.get(). Arrayen får **aldrig skickas eller returneras som en kopia**, utan ska alltid skickas som en **const char*** för att undvika onödig kopiering samt för att tydliggöra att namnet aldrig ska ändras efter det lästs in.

För testning

Det är viktigt att ni testar era spel ordentligt. Tänk då på att det ska vara lätt för er att testa. Ett sätt att göra det enklare att testa är att se till att den summa pengar spelaren börjar med är högre än maximumsatsningen på high stakes-bordet så att man direkt kan testa att det inte går att satsa mer än maximum, utan att man måste vinna en massa pengar innan det kan testas. Gör det både för er egen skull och för att det gör uppgiften enklare att rätta.

Inlämning

Gör mapp med uppgiftens namn. I denna mapp skapar ni ytterligare tvåmappar. En som heter **Exe** och en som heter **Source**.

I **Source**-mappen innehåller all källkod samt projekt filer.
Exe-mappen ska endast filer som behövs för att köra programmet ligga.
Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

För att lösa uppgiften är följande verktyg, eller koncept tillåtna. En del av er kanske redan har kunskaper i C++ och vet andra sätt att lösa uppgiften med hjälp av andra koncept. Det är inte tillåtet, uppgiften kan och skall lösas endast med angivna verktyg!

```
#include, #pragma once
assert
funktioner
namespace
char, short, int, long, float, double, bool, void, wchar_t, unsigned

class, struct, public, private
const, constexpr, static
enum class
std::array, referenser, pekare
typedef
if, else, else if, switch, while, do, for, return, break
random_device, mt19937, uniform_int_distribution
cout, cin
static_cast<>, const_cast<>
system(“cls”), system(“pause”)
```